# Demo

This demo requires R and Quarto Visit the instructions page to get ready.

## Demo I - Create your own report

### 1. Install additional dependencies and packages

Run the code below in the **R console** (it may take a couple of minutes)

```
# Install packages
install.packages(c('quarto', 'dplyr', 'ggplot2','kableExtra'))
```

### 2. Start a new Quarto document

#### Start a new QMD file

- In the Rstudio menu, go to `File > New File > Quarto document`.
- Keep the default options, fill in the filename text box
- **Save** the file with a filename ending `.qmd` in the project home directory (visible on the bottom right pane).
- Switch between the `visual` and `source` **tabs** of the R studio visual editor
- Edit the code in the **source** tab.

#### Edit the YAML header

Edit the YAML header delimited by (`---`). Add information about `author`, `affiliation` and `date` (use `last-modified` to always display the latest date) and define the output format as `pdf` or `html`. Beware of the *text indentation*! The header can be kept simple or it can contain a lot of information. An example:

```yaml
---
title: 'Title of the report or page'
subtitle: 'Subtitle'
date: last-modified # Display the date of the last edit
author:
  - name: 'Author 1 name'
    orcid: 1234-1234-1234-12345
    affiliations:
      - name: 'Institute, Faculty names'

  - name: 'Author 1 name'
    orcid: 1234-1234-1234-12345
    affiliations:
      - name: 'Institute, Faculty names'
format:
  html:
    toc: true
    toc-depth: 4
---
```

## 3. Create the report content

### Code chunks

The following code chunks have executable code to perform different actions on the data

> Tip 1: Code execution options
>
> The code chunks are delimited by ```` ```{r} ```` (you can specify other languages like Python, Julia, ObservableJS). For each code you can add *execution options* to the code chunks with `#|`, for example `#| echo: false` will not display the code. In Rstudio you can see the options available if you type `#|` and then press the TAB key on your keyboard . These options can be set globally for the entire document on the YAML header (under `execution:`).

### Load libraries

Let us first load all necessary libraries. Copy the code below inside a 'code chunk'

```r
# Load libraries
lapply(c('dplyr', 'ggplot2','kableExtra'), library, character.only = TRUE)
```

### Read the data

For this example we will use the data set 'Funding and Research Output Data' https://zenodo.org/records/5011201. The dataset is a table (CSV) with longitudinal data related to research outputs of researchers. Each researcher is identified with the variable `anonym_id` and there are multiple rows per researcher (*long format*).

We can read the table into a variable named *tbl* with the following code :

```
url <- 'https://zenodo.org/records/5011201/files/anonymised_funding_output_data.csv?download=
filename <- 'anonymised_funding_output_data.csv'
tbl <- read.csv(url, row.names = 1)
```

### Summary of the data

The data is longitudinal, we will summarize the cumulative sum of `n_articles_per_year` and `n_preprints_per_year` per researcher ( `anonym_id`).

```
data_summary <- summarise(
                group_by(tbl, anonym_id),
                total_articles = sum(n_articles_per_year, na.rm = TRUE),
                total_preprints = sum(n_preprints_per_year, na.rm = TRUE)
            )
```

### Table

We can render a table summarizing the data. For example:

```
#kableExtra::kable(head(tbl),format = "markdown")
kable(summary(data_summary)[,2:3],
      caption = paste0('Summary of table with ', nrow(data_summary), ' rows'),
      booktabs =T) %>% kable_styling(latex_options=c("striped"))
```

Table 1: Summary of table with 8793 rows

| total_articles | total_preprints |
|---|---|
| Min. : 0.00 | Min. : 0.000 |
| 1st Qu.: 5.00 | 1st Qu.: 0.000 |
| Median : 22.00 | Median : 0.000 |
| Mean : 45.78 | Mean : 3.356 |
| 3rd Qu.: 58.00 | 3rd Qu.: 2.000 |

Max. :991.00 Max. :252.000

### Plot

We can add a code chunk a plot how the number of preprints developed over time. Note that we add some code execution options for a caption and a label that we can use to reference to this figure in the text.

```
```{r}
#| label: fig-preprints
#| fig-cap: "Number of preprints over the years."

preprints <- summarise(
  group_by(tbl, year),
  n_preprints = sum(n_preprints_per_year)
  )

ggplot(preprints, aes(x= year, y = n_preprints)) +
  geom_point(shape=21, fill = "blue") +
  theme_minimal()
```
```

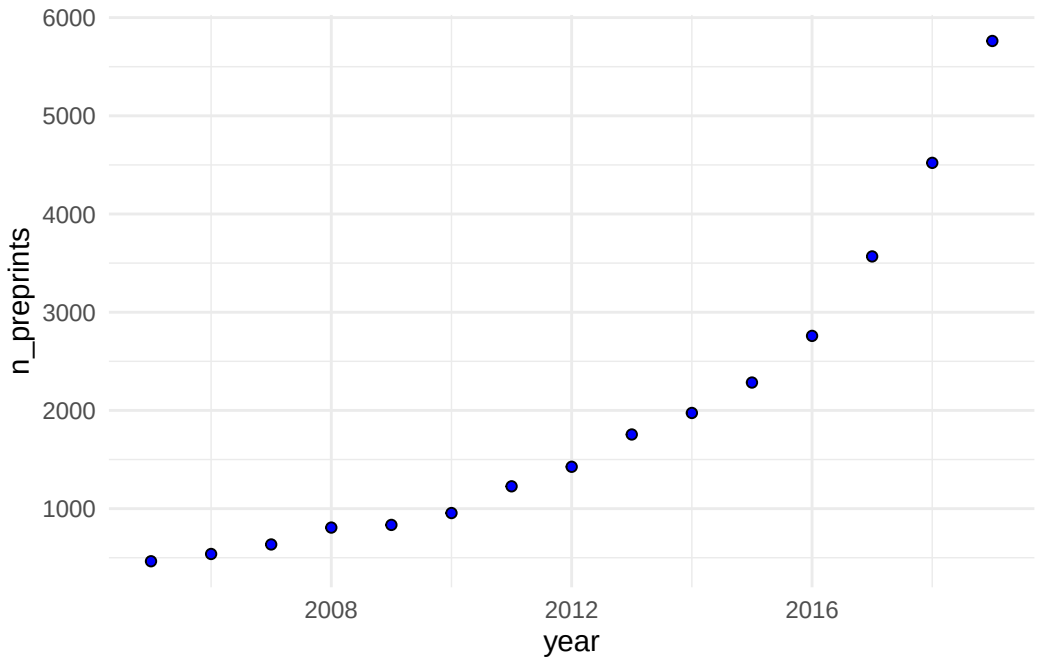



Figure 1: Number of preprints over the years.

I can now refer to the Figure 1 in the text using `[@fig-label]`.

### Inline code

Now we can use this data_summary and inline code to add a text describing the data using code to read some information from the table.

Use code snippets such as those below in your Markdown text to describe the data

Total number of researchers: `` `r count(data_summary)` ``

Maximum number of publications: `` `r max(data_summary$total_articles)` ``

Median number of publications: `` `r median(data_summary$total_preprints)` ``

### References

You can refer to a file with the reference list in the YAML header. In this example with have the .bib file in the same folder as this .qmd file:

```
bibliography: references.bib
```

Then use `[@authorDate]` to add a citation. For example, we recommend reading this manifesto for reproducible science (Munafò et al. 2017) or this nice easy-read entitled `cut with the tyranny of copy-pasting` (Perkel 2022) (yes, it is about R Markdown).

To get the best out of this functionality

- Connect your R studio project with a Zotero library
- Use the Visual editor in R studio and go to insert>citation

## 4. Render the report

Now that you edit the QMD file with your report:

- ☐ Check that the YAML header, the text and the code chunks are well delimited and you did not forget, for example, to close any chunk with (“‘).
- ☐ Click on the button `Render` with the blue arrow on top of the editor in R studio.
- ☐ A file with the same name as your .qmd file and the format specified in your header should have been created.

## 5. Refine the report

Try to explore different options for code execution:

- ☐ Hide all
- ☐ Print all code but do not execute it
- ☐ Fold all code (accessible by clicking on a button when saved an HTML)
- ☐ Hide the code reading the table, show the rest
- ☐ **Challenge** render the report as both an HTML document and as PDF

## Demo II - Some good habits in data visualisation

In the following we try illustrate some data visualisation principles by virtue of an example. TIt is based on the CRS Primer on Data Visualisation. The data is available here: https://osf.io/rk3mp. In addition we use ideas from Weissgerber et al. (2015).

The primer identifies five principles of data visualisation:

- Honesty
- Preattentive attributes
- Aesthetics
- Hierarchy
- Economy

We start from Figure 2, comparing the mean outcomes in two groups with different treatment. We first load additionally required R libraries, then the data.

```
library(tidyr)
library(readr)
library(ggbeeswarm)
```

```
vis_data <- read_csv("https://osf.io/download/rk3mp/")
```

```
data_summary <- vis_data %>%
  group_by(NameOfDrug) %>%
  summarise(value = mean(logOutc_value), sd = sd(logOutc_value))

p1 <- ggplot(data_summary, aes(x = NameOfDrug, y = value, fill = NameOfDrug)) +
  geom_bar(stat = "identity", alpha=0.7, width=0.5) +
  geom_pointrange(
    aes(ymin = value-sd, ymax = value+sd),
    color="gray40",
    linewidth = 1.3,
    size = 1.5
    ) +
  theme(
    panel.grid = element_line(color="lightgrey"),
    legend.title = element_text(size = 13),
    legend.text = element_text(size = 11)
```

```
  ) +
  labs(y = "Outcome", x = "Drug") +
  scale_fill_manual(values = c("red", "green"), name = "Drug") +
  coord_cartesian(ylim = c(7, 9.5)) +
  scale_y_continuous()

p1
```

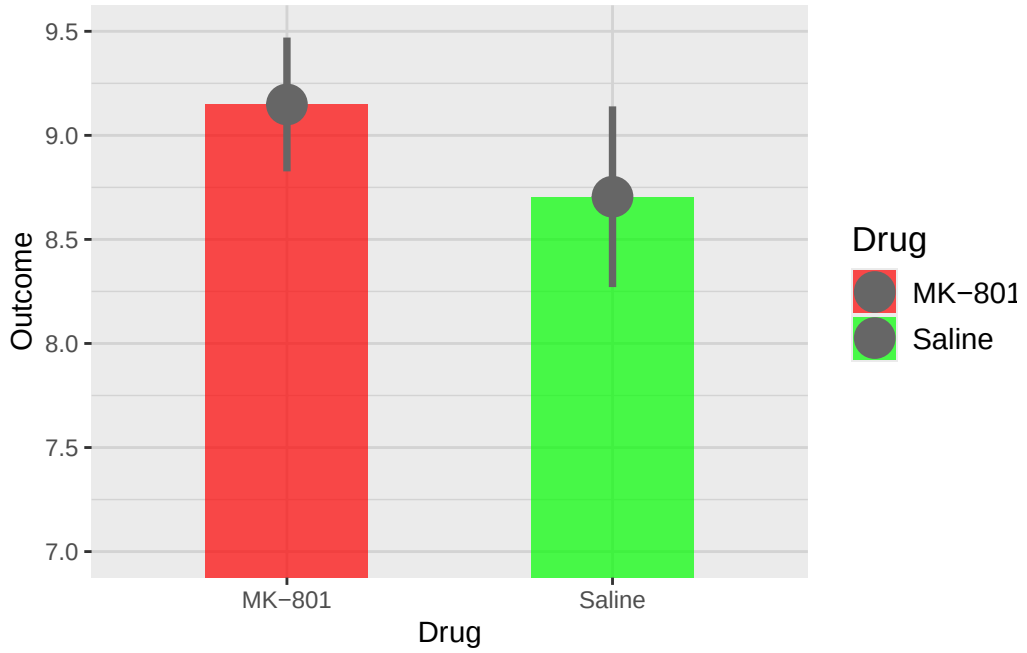



Figure 2

### Improve the bar plot

We can improve this plot along some of the data visualisation principles introduced above.

- Economy:
  - set a minimalist theme
  - remove the redundant legend
  - remove the panel grid
  - remove axis.ticks
- Preattentive attributes
  - use more easily distinguishable colors

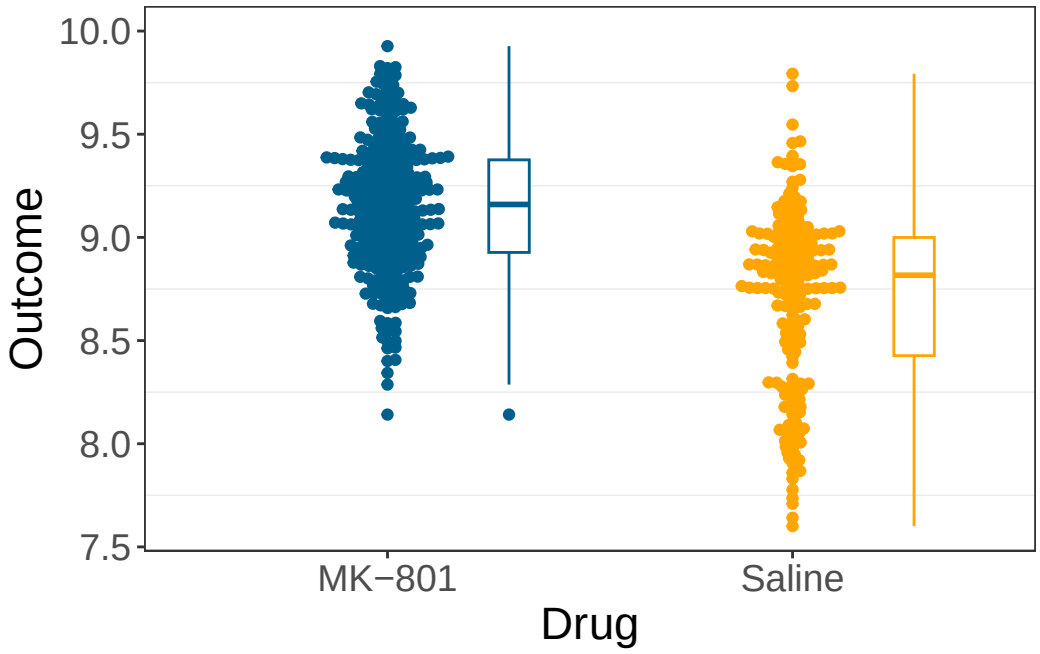
    - A simulator for inspecting color palettes under different color vision conditions is available here: https://www.susielu.com/data-viz/viz-palette
    - Zeileis et al. (2020) provides a comprehensive discussion of using colors in R.

- Honesty:
  - set axis to full range
  - display aspects of the data distribution, not only summary statistics
    - use boxplot instead of barplot
  - **display raw data**

- Aesthetics:
  - use beeswarm or jitter
  - color only the shapes' borders (use the aesthetic 'color' instead of 'fill')
  - set appropriate axis font sizes

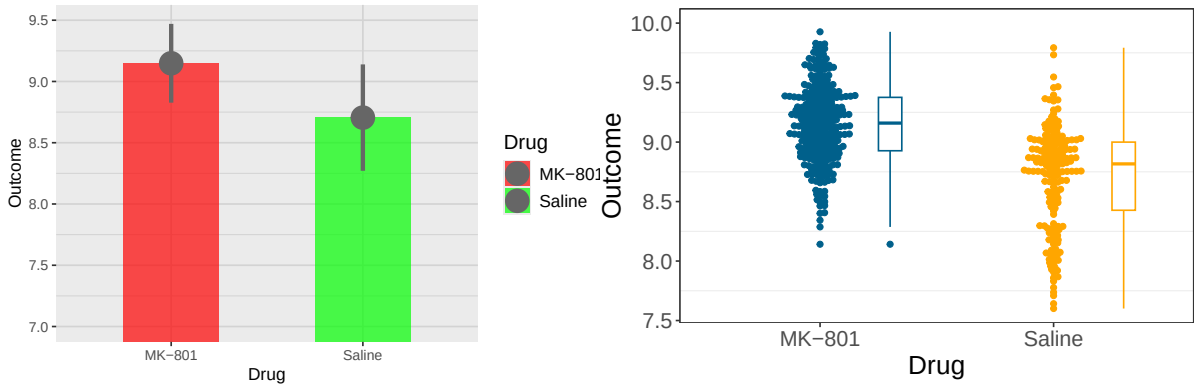
```
p2 <- ggplot(vis_data, aes(x=NameOfDrug,y=logOutc_value,color=NameOfDrug))+
  geom_boxplot(width=0.1,position = position_nudge(x=0.3))+
  geom_beeswarm()+
  theme_bw()+
  theme(
    panel.grid.major = element_blank(),
    axis.text = element_text(size=14),
    axis.title = element_text(size=17),
    legend.position = "none"
  )+
  scale_color_manual(values=c("#00608b","#ffa600"),name="Drug")+
  labs(y="Outcome",x="Drug")+
  scale_y_continuous(limits=c(7.6,10),breaks=seq(7.5,10,0.5))

p2
```




### Putting the two graphs together

We can use Quarto functionality to display the original and the revised plot side-by-side, specifically the execution options `layout-ncol` to define the number of columns, `fig-cap` for the global caption, and `fig-subcap` for column-specific captions.




(a) Original figure

(b) New figure

Figure 3: Two figures that display the same data.

## Demo III - Explore other examples

The are many examples of dynamically generated reports online. Try exploring the links below.

- Example 1. Reproducible report in HTML using R Markdown (SNSF report: https://data.snf.ch/stories/open-research-data-2023-en.html)
- Example 2. Reproducible report in pdf using Latex (robustness check analysis: https://gitlab.uzh.ch/fabio.molo/nhb-replication.)
- Example 3. Website with course materials: https://github.com/avehtari/BDA_course_Aalto/tree/master
- Example 4. Reproducible slides with Quarto: https://github.com/kjhealy/flipbookr-quarto/tree/main?tab=readme-ov-file.

Try to reflect about the following questions:

- ☐ Can you easily find the repository with the source files?
- ☐ What dynamic reporting tool they use?
- ☐ Can you access the output, rendered documents easily ?
- ☐ Do they use some advanced features for dynamic reporting or computational reproducibility?
- ☐ What would be the workflow to reproduce those reports?

### references


Munafò, Marcus R., Brian A. Nosek, Dorothy V. M. Bishop, Katherine S. Button, Christopher D. Chambers, Nathalie Percie Du Sert, Uri Simonsohn, Eric-Jan Wagenmakers, Jennifer J. Ware, and John P. A. Ioannidis. 2017. "A Manifesto for Reproducible Science." *Nature Human Behaviour* 1 (1): 0021. https://doi.org/10.1038/s41562-016-0021.

Perkel, Jeffrey M. 2022. "Cut the Tyranny of Copy-and-Paste with These Coding Tools." *Nature* 603 (7899): 191–92. https://doi.org/10.1038/d41586-022-00563-z.

Weissgerber, Tracey L., Natasa M. Milic, Stacey J. Winham, and Vesna D. Garovic. 2015. "Beyond Bar and Line Graphs: Time for a New Data Presentation Paradigm." *PLOS Biology* 13 (4): e1002128. https://doi.org/10.1371/journal.pbio.1002128.

Zeileis, Achim, Jason C. Fisher, Kurt Hornik, Ross Ihaka, Claire D. McWhite, Paul Murrell, Reto Stauffer, and Claus O. Wilke. 2020. "**Colorspace** : A Toolbox for Manipulating and Assessing Colors and Palettes." *Journal of Statistical Software* 96 (1). https://doi.org/10.18637/jss.v096.i01.